A STUDENT AND PRESENTER GUIDE

RAG Eval Lab

Architecture, Streamlit and Windows setup

A local, evidence-first laboratory for understanding **Retrieval-Augmented Generation (RAG)** and evaluating its answers with **DeepEval**. This guide describes the implemented project, its important components, and a repeatable demonstration workflow.

> **The central idea**
> Do not judge a chatbot only by how fluent it sounds. Inspect the question, retrieved evidence, answer, and evaluation reasons separately.

Project at a glance	Current implementation
Application	Python package + Streamlit website + command-line runner
Data	9 synthetic policy documents; 14 cases (9 development / 5 holdout)
Retrieval	Local TF-IDF and cosine similarity; no semantic embedding model
Models	Ollama generator and judge roles; default qwen2.5:7b
Outputs	Inspectable runs, DeepEval scores/reasons/errors, JSON and CSV

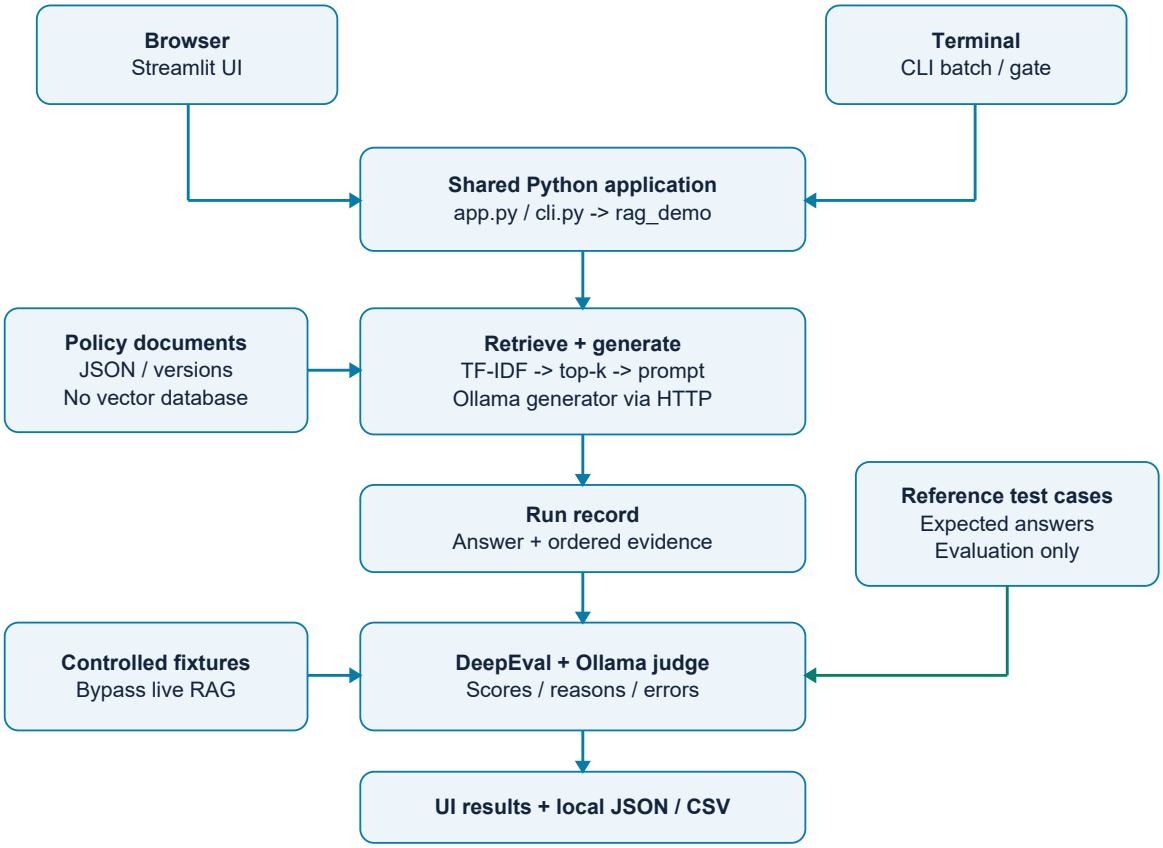
Read this guide

Architecture and application	Setup and operation
02 Architecture and boundaries	08 Windows setup
03 Components and data contracts	09 CLI, persistence and comparison
04 Retrieval and generation	10 Troubleshooting
05 DeepEval evaluation	11 Validation and limitations
06 How Streamlit works	12 Teaching plan and references
07 Website walkthrough	All commands use Windows PowerShell.

Scope: source-grounded architecture, not a model benchmark. No example metric scores are presented as measured results. All company policies are fictional.

02 / Architecture and boundaries

One Python application, two entry points, one local model service.



Logical data flow. Generator and judge are roles served by Ollama; they need not be different model installations. Evaluation can be run after generation.

Deployment on your machine

Boundary	What runs there
Browser	Streamlit UI, normally http://127.0.0.1:8501. The browser does not run the LLM.
Python process	Streamlit or the CLI invokes shared modules directly. No separate REST backend, message queue, or background evaluation worker is implemented.
Ollama service	Normally http://localhost:11434. /api/tags checks installed models; /api/chat handles generation. DeepEval's Ollama adapter serves judge calls.
Local disk	Packaged policy JSON, installed model files managed by Ollama, and project result files. No database server is required.

Keep evaluation references out of generation
The generator receives the question and retrieved policy passages, not the expected answer or expected source IDs. References enter evaluation and diagnostics only.

03 / Components and data contracts

Source paths below are relative to the project root.

Component	Responsibility
app.py	Streamlit widgets, actions, run inspection, comparisons and downloads.
rag_demo\pipeline.py	Load corpus, filter/rank evidence, call Ollama, create live runs and hand-authored controlled runs.
rag_demo\evaluation.py	Build DeepEval test cases, run local judge metrics, record errors and calculate source-ID coverage.
rag_demo\models.py	Pydantic schemas and validation for the corpus and run records.
rag_demo\storage.py	JSON save/load, CSV flattening, regression-gate checks.
rag_demo\cli.py	Batch selection, model preflight, evaluation and exit codes.
rag_demo\data\policies.json	Synthetic documents and evaluation references.
.streamlit\config.toml	Loopback binding, dark theme and usage-statistics opt-out.
pyproject.toml / requirements-lock.txt	Pinned runtime dependencies, CLI entry point, test/lint settings and full installed dependency snapshot.

The records passed between components

Record	Important fields
Document	id, title, version, current, text
Case	question, expected_answer, expected_source_ids, category, split, answerable
Corpus / Passage	Corpus holds documents and cases. Passage holds a document and retrieval score; list order preserves ranking.
Run	ID, timestamp, case, mode, configuration, answer, passages, generator, k, corpus SHA-256, timings, judge and metric results.
MetricResult	Name, score, threshold, passed, reason, error, duration. Errors have no score or pass/fail value.

Validation rejects duplicate document/case IDs, invalid expected-source links, inconsistent answerability, unknown record fields and invalid metric scores. The corpus fingerprint covers both documents and reference cases, helping identify dataset changes.

04 / Retrieval and generation

The live path is deliberately small enough to inspect end to end.

1. Load and select. Load the packaged JSON into validated records. The website selects a dataset question and one or more configurations; it is not a free-form chat interface.

2. Filter sources. Baseline permits all 9 documents. The other configurations permit the 8 current documents only. Filtering occurs before ranking.

3. Represent text. scikit-learn TF-IDF vectorizes each document's title plus text, using English stop words and one- and two-word features. The question uses the same vectorizer.

4. Rank and select. Cosine similarity ranks documents. Up to k positive-score matches are returned; k defaults to 3. No overlap can yield an empty evidence list.

5. Build the prompt. Include source IDs, version/current status and passage text, followed by the question. Improved instructions add grounding, conditions, citations and abstention.

6. Generate and record. Call Ollama synchronously with temperature 0 and streaming off. Create a Run preserving the exact evidence. Empty output and HTTP failures are errors.

Configuration	Sources	Prompt
baseline	Current + superseded	Simple policy assistant
retrieval-improved	Current only	Same simple prompt
improved	Current only	Grounding, approval exceptions, citations, missing-information rule

The hotel example

Question: Can I claim a hotel costing INR 8000 per night?
Evidence: the standard limit is INR 6000; higher costs need prior written manager approval.
Supported answer: INR 8000 can be reimbursed with that approval; without it, reimbursement is capped at INR 6000.

> **What this retriever is not**
> Each short document is one passage. There is no PDF ingestion, chunking service, neural embedding model, vector database or reranker. The TF-IDF matrix is rebuilt per retrieval call. These simplifications are intentional for a teaching corpus.

Controlled fixtures take a different route

Six fixtures pair hand-authored answers and evidence: good, retrieval-miss, unsupported, off-topic, outdated and bad-abstention. They bypass retrieval and generation, use mode 'controlled', and report 0.00s generation time. They start unscored but can be evaluated.

05 / DeepEval evaluation

Evaluation is a separate workflow, not part of the generator's prompt.

Run field	DeepEval LLMTestCase field
case.question	input
answer	actual_output
case.expected_answer	expected_output
Ordered passage text	retrieval_context

Metric	Question it answers	Reference?
Answer relevancy	Is the response relevant to the question?	No
Faithfulness	Does the answer align with supplied evidence?	No
Contextual recall	Does the evidence support the facts needed for the reference answer?	Yes
Contextual precision	Are useful passages ranked ahead of irrelevant passages?	Yes
Policy correctness (G-Eval)	Does the answer satisfy current-policy facts, limits and essential conditions?	Yes
Abstention (G-Eval)	For an unanswerable case, does it acknowledge missing information without inventing facts?	Yes

Execution and special cases

The code constructs an explicit OllamaModel judge, then calls each metric's measure() sequentially with async_mode=False. A metric may make several local model calls. Each measured result retains its reason, score, threshold and elapsed time.

Answerable cases use the four standard metrics plus policy correctness, except that faithfulness is omitted if there is no evidence. Unanswerable cases omit contextual recall and precision and use the abstention rubric. Model/parse failures are stored as explicit metric errors, never disguised as zero scores or passing results.

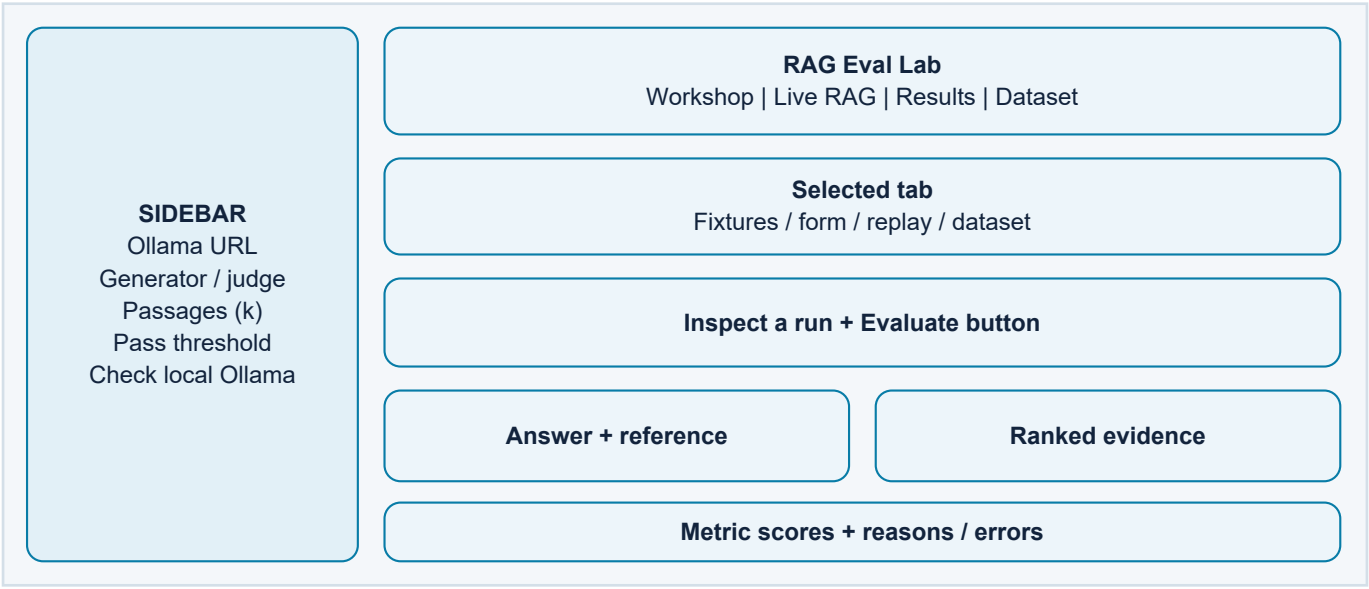
> **Interpret scores with care**
> The default 0.7 threshold is illustrative. A score of 0.9 is not a 90% probability of correctness. Relevant answers can be wrong; answers faithful to outdated documents can violate current policy. Read reasons and calibrate the judge with human review.

Source-ID coverage is a separate deterministic diagnostic: retrieved expected IDs divided by all expected IDs. It is not DeepEval contextual recall, and is not reported when no supporting source IDs are expected.

06 / How Streamlit works

A Python-driven web interface, not a separate JavaScript application.

Streamlit turns Python calls into browser controls, tables, messages and charts. Running the launch command starts a local Python server; the browser connects to it. app.py calls the shared Python functions directly.



Schematic of the implemented layout; not a screenshot or measured result.

The execution model

Reruns. Normal widget interactions trigger script reruns. Button conditions and the live-generation form prevent every rerun from generating answers or judging them.

Forms. The question and configuration controls are inside st.form. Generate answers submits those choices together. The dataset split selector sits outside the form.

Session state. st.session_state keeps the in-memory history across reruns in the current browser session. A new session starts with an empty history; saved JSON can be loaded.

Persistence. remember() replaces the same run ID in history, saves the history to results\session.json, and updates state. Re-evaluating the same run replaces its metric results.

Rendering. Tabs organize the page; columns show answer and evidence side by side; expanders show references and judge reasons; download buttons export the results.

> **Operational boundary**
> Generation and evaluation run synchronously in the app process, not in a job queue. Keep the Streamlit terminal open. Session state is per session, but results\session.json is a shared filename: this is a single-presenter demo, not a multi-user persistence design.

07 / Website walkthrough

Follow this order when explaining the project to students.

Screen / action	What to explain
Sidebar	Ollama URL points to the local service. Generator writes answers; judge assesses them. k controls evidence count. Threshold controls the illustrative cutoff. Check local Ollama confirms service reachability and lists models.
1. Workshop	Select a controlled scenario, then click Load controlled example. Read the controlled-fixture label before discussing its answer.
2. Live RAG	Choose split, dataset question and configurations; click Generate answers. Inspect the actual evidence before judging the answer.
Run inspector	Inspect a run selects the record to display and evaluate. Read its case, configuration, mode and ID; expand the reference answer.
Evaluate button	Preflight checks the judge model, then scores the selected run. Read each score and reason; investigate errors separately from quality failures.
3. Results & replay	Load one or more saved files, compare matching live runs, and download JSON/CSV. A replay is not a fresh evaluation.
4. Dataset	Inspect all 14 cases and all 9 policy documents. Show development/holdout splits and current/superseded source labels.

Read the labels correctly

Label	Meaning
CONTROLLED FIXTURE	Hand-authored answer and evidence; not naturally observed model output.
NOT EVALUATED	This run has no DeepEval results yet.
Evaluation completed and saved locally	The evaluation finished without recorded metric errors. It does **not** mean all metrics passed or the answer is correct.
PASS / FAIL / ERROR	Threshold met / threshold not met / evaluation could not produce a valid result. These are different states.

Scenario selection is not run selection
If the top selector says 'Answer about meals instead of hotels' but Inspect a run says 'unsupported', the unsupported run is still being inspected. Click Load controlled example and confirm the desired run before evaluating.

Presenter line: 'The examiner finished marking' is different from 'the student passed'. Scroll to the metric results before drawing conclusions.

08 / Windows setup

Run these commands in PowerShell from the project directory.

Prerequisites: Python 3.11+ (the project was built with 3.11.9), the project files, and network access for package/model downloads. Install Ollama separately for live generation or scoring. The Python ollama package does not install the Ollama desktop/server service.

1. Create an isolated project environment

```
cd C:\Temp\RAGEvalDemo
python --version
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
```

If the existing environment is already installed, reuse it; do not recreate it unnecessarily. Using the executable path avoids PowerShell activation-policy issues.

Alternative reproducible install: in a fresh environment, install requirements-lock.txt, then install the project without resolving dependencies again.

```
.\.venv\Scripts\python.exe -m pip install -r requirements-lock.txt
.\.venv\Scripts\python.exe -m pip install --no-deps -e .
```

2. Start the website

```
.\.venv\Scripts\python.exe -m streamlit run app.py
```

Open **http://127.0.0.1:8501**. Keep the terminal open; Ctrl+C stops the server. Controlled examples work now without Ollama. To use the older demo URL on port 8517, add `--server.port 8517` to the launch command.

3. Install and prepare the local models

Install Ollama for Windows, open it, and then open a new PowerShell terminal so PATH is refreshed.

```
ollama pull qwen2.5:7b
ollama list
Invoke-RestMethod http://localhost:11434/api/tags
```

If the service is not running, use `ollama serve` in a separate terminal. Do not start a second server if the address is already in use. The model download is several GB; memory needs and latency depend on model, quantization and hardware.

> **4. Verify before evaluating**
> In the sidebar, use http://localhost:11434 and the exact installed judge name qwen2.5:7b. Click Check local Ollama. Load a fixture, then Evaluate this run with local DeepEval. No OpenAI key or Confident AI login is needed for this path.

09 / CLI, persistence and comparison

Use the CLI for repeatable batches; use the website to inspect evidence.

The installed `rag-demo` entry point invokes the same pipeline and evaluation modules as Streamlit. The examples below use a short PowerShell variable.

```
$demo = ".\.venv\Scripts\rag-demo.exe"

# No model required: six explicitly unscored fixtures
& $demo --fixtures --output results\fixtures.json

# Local judge required: evaluate the controlled examples
& $demo --fixtures --evaluate --output results\scored-fixtures.json

# Actual live pipeline: same case, two configurations
& $demo --evaluate --configuration baseline --case hotel-exception `
  --output results\baseline.json
& $demo --evaluate --configuration improved --case hotel-exception `
  --output results\improved.json

# Evaluate holdout cases and enforce thresholds
& $demo --evaluate --gate --split holdout --output results\holdout.json
```

PowerShell continuation uses a backtick at the end of the line, with no trailing spaces. Use --configuration retrieval-improved to isolate source filtering. --generator, --judge, --url, --k and --threshold override defaults.

Exit code	Meaning
0	Completed; if --gate was supplied, all recorded metrics passed the gate.
1	Evaluation completed but the gate failed.
2	Invalid input, model/metric error or persistence failure.

What is persisted

JSON preserves the question/reference, answer, ordered evidence, configuration, models, k, thresholds, run timestamp, corpus fingerprint, timings and metric details. CSV flattens metrics for spreadsheet analysis. JSON is saved through a temporary file and atomic replacement.

The CLI saves completed runs after every case and writes CSV at the end. Use distinct output names: the default results\latest.json is overwritten. The UI writes results\session.json. Reload saved files explicitly after starting a new browser session.

> **Fair comparisons and history**
> The comparison view pairs baseline/improved live runs with matching case, corpus, generator, judge, k and metric threshold; it excludes fixtures and metric errors. The corpus hash is not a code/model-weight fingerprint. Export a run before re-evaluating it if you want to retain its earlier scores.

10 / Troubleshooting

Diagnose service availability before judging answer quality.

Symptom	Likely cause and next action
WinError 10061 / connection refused	Nothing is listening at the configured Ollama URL/port, or the address is wrong. Open Ollama or run ollama serve. Check /api/tags and the sidebar URL. This is not a low evaluation score.
Model missing	Use ollama list, pull the required model, and copy its exact name/tag into the judge or generator setting.
ollama is not recognized	Install the separate Ollama application; reopen PowerShell after installation. pip installing the Python client alone is insufficient.
Website will not open	Keep the Streamlit process running. Use the URL/port printed in its terminal. A browser tab can remain open after its server stops.
Address already in use	A service may already be running. Test its health before starting another. For Streamlit, choose a different --server.port if necessary.
Evaluation is slow / times out	A metric can call the judge repeatedly, sequentially. Warm up the model, reduce the number of cases for the live demo, and rehearse a suitable model for the available hardware. Inspect timeout errors rather than hiding them.
Metric ERROR / invalid JSON	The judge may not return valid structured output. Inspect the error, confirm model/version, and use a more capable compatible local judge. Do not replace errors with passing scores.
Wrong example is displayed	Load the selected fixture, then check Inspect a run. The scenario selector and run inspector are separate controls.
No paired comparison	Evaluate both live baseline and improved runs for the same case and matching settings. Load both saved files. Controlled runs are intentionally excluded.
Generation time is 0.00s	Expected for a hand-authored controlled fixture. Evaluation has its own duration; no live answer was generated for that fixture.

Two independent health checks

```
# Streamlit website, using its actual launch port
Invoke-WebRequest http://127.0.0.1:8501/_stcore/health

# Ollama service and installed models
Invoke-RestMethod http://localhost:11434/api/tags
```

> **Completion is not correctness**
> A green 'Evaluation completed and saved locally' message means the scoring process finished without recorded metric errors. Look below it for PASS/FAIL and judge reasons.

11 / Validation and limitations

Software correctness and model quality require different evidence.

Run the existing validation tools

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe -m ruff check .
.\.venv\Scripts\python.exe -m mypy rag_demo app.py
.\.venv\Scripts\python.exe -m pip check
```

Test area	What is checked
Pipeline / schemas	Dataset integrity, source filtering/ranking, exact context sent to the model, empty outputs and HTTP failures.
Evaluation / storage / CLI	Metric field wiring, explicit errors, persistence, CLI argument validation, nonzero gates and rejection of unscored runs.
Streamlit	Fixture labels, missing-service messages, saved replay and comparison pairing using Streamlit's application test interface.
Real metric protocol	Actual DeepEval metric classes consume synthetic Ollama structured responses while socket connections are blocked. This tests integration, not real judge intelligence.

Local-first behavior

Model URLs are limited to loopback HTTP. DeepEval telemetry and Streamlit usage statistics are disabled. Evaluation uses standalone metric measurement rather than a hosted test-run upload workflow. Package/model installation needs network access; inference need not use a hosted provider. This is not a network sandbox: a locally configured forwarding proxy could still send requests elsewhere.

Important limits to explain

Small, inspectable corpus. Nine documents and fourteen cases demonstrate concepts, not production accuracy or statistical significance. Do not tune on the five holdout cases.

Judge uncertainty. Temperature 0 does not guarantee reproducibility or correct scoring. Audit reasons, compare with human labels, and repeat representative cases.

Evidence and citation limits. Faithfulness is not current-policy truth. Citation output is requested by the prompt but citation accuracy is not separately verified.

Single-user operation. No authentication, multi-user isolation, concurrency-safe history merge, durable queue or production observability layer is implemented.

> **Possible next steps - not implemented**
> After establishing a baseline: add a larger reviewed dataset, persistent indexing or semantic retrieval, explicit citation checks, judge calibration and multi-user storage. Measure each change separately; do not assume extra infrastructure improves quality.

12 / Teaching plan and references

Explain the experiment first, then show its implementation.

A one-minute explanation
'Our website makes the question, evidence, answer and evaluation visible. The retriever finds policy passages; a local model writes an answer; DeepEval asks a local judge to assess it. We save the results to compare changes. Relevant does not mean correct, and faithful to an outdated document does not mean correct under today's policy.'

Minutes	Demonstration
0-5	Load unsupported. Invite students to assess the INR 8000 hotel answer.
5-12	Reveal the two policy passages; distinguish retrieval from generation.
12-20	Explain relevancy, faithfulness, recall, precision and policy correctness.
20-30	Evaluate 2-3 examples; read reasons. Use labelled saved replay if latency is high.
30-40	Contrast missing evidence, unsupported claims, outdated sources and abstention.
40-49	Compare the same live question under baseline, retrieval-improved and improved.
49-55	Run a CLI gate. Inspect a failing metric or explicit judge error.
55-60	Discuss holdouts, judge uncertainty and what the evidence does not prove.

Student exercise

Predict the needed hotel-policy sources, run baseline, change only source filtering, then add the grounded prompt. Record source IDs, answer, scores, reasons and latency. Explain which component changed the result. Repeat with an unanswerable question; do not invent scores or claim universal improvement from one example.

Source and version notes

This guide was checked against app.py, rag_demo modules, the dataset, tests, README.md, pyproject.toml and .streamlit\config.toml on 7 November 2026. Project version: 0.1.0. Runtime pins: DeepEval 3.9.9; Ollama Python client 0.6.3; Streamlit 1.66.1; scikit-learn 1.9.1; httpx 0.28.1; Pydantic 2.14.2. The Ollama server and model installation are separate. Check the project files again after upgrades.

Official references

Streamlit documentation - https://docs.streamlit.io/

Ollama installation - https://ollama.com/download/windows

DeepEval Ollama integration - https://deepeval.com/integrations/models/ollama

DeepEval RAG evaluation guide - https://deepeval.com/guides/guides-rag-evaluation

DeepEval G-Eval - https://deepeval.com/docs/metrics-llm-evals